A3-PS022-TC369

eXBP/XDP Rate limiting for DNS Query Floods in a Lab

Ahmad Yaseen A
ahmadyaseen@karunya.edu.in

Aaron Paul Santhosh
aaronpaul25@karunya.edu.in

AIORI Hackathon 3
October 12, 2026

Problem Description

Core Issue: High-volume UDP DNS query floods overwhelm Linux servers by forcing the kernel to waste CPU resources on memory allocation before packets are even processed.

Key Issues:

- **The Kernel Bottleneck (SoftIRQ Saturation):**
  - In standard Linux networking, when a packet arrives, the kernel allocates a **`sk_buff`** (socket buffer) in memory *before* any filtering occurs.
  - During a UDP DNS flood (e.g., 100,000 packets/sec), the CPU spends ~60-80% of its cycles on **`ksoftirqd`** threads managing these buffers, leaving insufficient cycles for legitimate application processing.
- **Inefficiency of Netfilter (nftables/iptables):**
  - Traditional firewalls operate at the **`NF_INET_PRE_ROUTING`** hook, which is *after* memory allocation.
  - **The Cost:** Even if a packet is dropped by **`nftables`**, the CPU has already paid the cost of copying data from the NIC DMA ring to RAM and allocating kernel memory. This is "wasted compute."

Problem Description

- **Vulnerability of UDP/DNS:**
  - **Connectionless Nature:** UDP lacks a handshake, making it trivial to spoof source IPs (IP Spoofing).
  - **Amplification Factor:** DNS responses can be 50x larger than queries. Attackers exploit this to create Reflection/Amplification DDoS attacks, saturating bandwidth and server CPU simultaneously.

- **Impact on Service Availability:**
  - Legitimate users experience high latency (jitter) or timeouts because the DNS server's thread pool is exhausted handling garbage traffic.

Background

1.The Threat: UDP & DNS Vulnerabilities

- **Protocol Nature:** DNS primarily uses UDP for speed, but its connectionless nature lacks a handshake mechanism.
- **Spoofing & Amplification:** Attackers easily spoof source IPs and exploit the 50x amplification effect (small query → large response) to launch Reflection/Amplification DDoS attacks.
- **Impact:** These floods saturate network bandwidth and overwhelm server resources, causing service outages for legitimate users.

2.The Technical Gap: The Kernal Bottleneck

- **Late Intervention:** Traditional firewalls (nftables/iptables) operate after the kernel has already allocated memory (socket buffers) for the packet.
- **Wasted Resources:** During a flood, the CPU spends significant cycles allocating memory for malicious packets that will eventually be dropped.
- **Performance Degradation:** This "softirq" overhead leads to CPU saturation and high latency, even for legitimate traffic, because the system is busy processing garbage.

Background

3. The Need for Early-Stage Filtering

- **Current Limitations:** Existing solutions are either too slow (user-space processing) or too complex (heavy kernel modifications).
- **The Opportunity:** There is a critical need for a lightweight, kernel-level mechanism that can inspect and drop packets before they consume system resources.
- **Why eBPF/XDP?** eBPF provides a safe, sandboxed environment to run custom logic in the kernel, while XDP allows this logic to run at the NIC driver level—the earliest possible point in the network stack.

Literature & RFC Review

- **1. Academic Studies on eBPF Security:**
  - **Kostopoulos et al. (2026):** *Systematic Literature Review on eBPF Cybersecurity.* Analyzed 54 peer-reviewed studies.
    - *Key Finding:* eBPF-based DDoS mitigation achieves a **median CPU overhead of only 2.4%**, compared to significant overhead in user-space solutions.
  - *Accuracy:* Detection accuracy ranges from **94-99%** for signature-based and rate-limiting filters.

.

Literature & RFC Review

- **2. Industry Standards (RFCs):**
  - **RFC 768 (UDP):** Highlights the lack of connection establishment, confirming the vulnerability to spoofing.
  - **RFC 5382 (DNS Challenge-Response):** Discusses application-layer defenses (like DNS Cookies). While effective, these add latency and complexity to the DNS server, motivating the need for **network-layer** (XDP) pre-filtering.
- **3. Existing Proposals:**
  - **Cilium/Isovalent:** Industry leader in eBPF networking. Their whitepapers confirm that XDP is the de facto standard for high-performance, low-latency security policies in cloud-native environments.

Proposed Solution

eBPF/XDP Rate Limiting Architecture

1. Overview of the Solution

The proposed solution addresses the vulnerability of Linux servers to high-volume UDP DNS query floods by implementing a custom **eBPF/XDP (eXpress Data Path) rate-limiting filter**. Unlike traditional firewall solutions that operate within the kernel networking stack, this solution deploys an eBPF program directly into the network interface driver. By intercepting packets at the earliest possible stage—before they are copied into kernel memory—the system ensures that malicious traffic is discarded with minimal CPU and memory overhead, thereby preserving system resources for legitimate DNS queries.

Proposed Solution

2. Core Mechanism: Early-Stage Packet Filtering

The cornerstone of this architecture is the **XDP Hook**, which attaches the eBPF program to the **`eth0`** network interface of the target Linux host.

- **Pre-Kernel Execution:** The eBPF bytecode executes within the NIC driver's interrupt context. This positioning allows the program to inspect and process packets before the Linux kernel allocates a **`sk_buff`** (socket buffer) or triggers expensive softirq handlers.
- **Zero-Copy Drop:** When a packet is identified as malicious, the program returns the **XDP_DROP** verdict. This action discards the packet immediately at the driver level, preventing any further processing by the kernel. This "zero-copy" approach eliminates the CPU cycles typically wasted on memory allocation and context switching for dropped packets.

Proposed Solution

3. Bounded UDP/DNS Parsing

To ensure safety and stability, the solution implements bounded parsing for all incoming packets.

- **Safe Header Inspection:** The eBPF program strictly validates packet structures by checking `data_end` pointers before accessing any header fields. This prevents out-of-bounds memory access, which could otherwise lead to kernel panics or verifier rejections.
- **Protocol Identification:** The parser sequentially validates the Ethernet, IP, and UDP headers. It specifically checks for the Internet Protocol (`IPPROTO_UDP`) and verifies that the destination port is **53** (the standard port for DNS). Only packets matching these criteria proceed to the rate-limiting logic; all other traffic is passed through to the kernel via `XDP_PASS` to ensure no legitimate non-DNS traffic is disrupted.

Proposed Solution

4. Dual-Layer Rate Limiting Logic

The solution employs a two-tiered rate-limiting strategy to handle both concentrated and distributed attacks effectively. This logic utilizes two distinct eBPF Map types for optimal performance:

- **Per-Source Rate Limiting (Hash Map):**
  - **Implementation:** A **BPF_MAP_TYPE_HASH** is used to maintain a dynamic counter for each unique source IP address.
  - **Function:** As DNS queries arrive, the program looks up the source IP in the hash map. If the query count for that specific IP exceeds a predefined threshold (e.g., 200 queries per second), the packet is dropped.
  - **Benefit:** This granular approach isolates malicious actors. If one IP address launches a flood, only that specific source is penalized, allowing other legitimate users to continue accessing the service without interruption.

Proposed Solution

- **Global Rate Limiting (Array):**
  - **Implementation:** A **BPF_MAP_TYPE_ARRAY** is used to track the aggregate traffic volume across all sources.
  - **Function:** This counter increments for every valid DNS query that passes the per-source check. If the total global count exceeds a system-wide capacity limit (e.g., 5,000 queries per second), subsequent packets are dropped regardless of their source IP.
  - **Benefit:** This layer protects the server against distributed denial-of-service (DDoS) attacks where many different IP addresses send moderate amounts of traffic. Individually, these sources may appear benign, but collectively they threaten to saturate the server's resources.

Proposed Solution

5. Traffic Classification and Outcome

The final decision logic classifies each packet into one of two categories:

- **Malicious Traffic (XDP_DROP):** Packets that violate either the per-source or global rate limits are immediately discarded. This prevents the kernel from consuming resources on garbage data.
- **Legitimate Traffic (XDP_PASS):** Packets that adhere to all rate limits are passed to the standard Linux kernel network stack. These packets undergo normal socket buffer allocation and are delivered to the DNS server application for resolution.

Optimization Proposed

List of Optimizations & Enhancements

1. Bounded Header Parsing with Safety Guards

- **Enhancement:** Implementation of strict bounds checking (**`data_end`**) for every header access (Ethernet, IP, UDP).
- **Benefit:** Prevents kernel panics caused by malformed packets. Ensures the eBPF program complies with the strict verifier constraints, guaranteeing system stability during high-load stress tests.

2. Dual-Layer Rate Limiting Architecture

- **Enhancement:** Combining **Per-Source Limiting** (via **BPF_MAP_TYPE_HASH**) with **Global Limiting** (via **BPF_MAP_TYPE_ARRAY**).
- **Benefit:** Addresses both **concentrated attacks** (single IP flood) and **distributed attacks** (botnets). The per-source limit protects individual users from being blocked by one attacker, while the global limit protects the server from total saturation.

Optimization Proposed

List of Optimizations & Enhancements

3. Zero-Copy Packet Drop at NIC Driver Level

- **Enhancement:** Utilizing the XDP hook to return **XDP_DROP** before the kernel allocates a **`sk_buff`** (socket buffer).
- **Benefit:** Eliminates the CPU overhead of memory allocation (**`kmalloc`**) and context switching for malicious packets. This is the primary driver for the significant reduction in CPU utilization compared to nftables.

4. Low-Latency Bytecode Execution

- **Enhancement:** Replacing complex kernel tree traversals (used in Netfilter) with direct eBPF bytecode execution.
- **Benefit:** Reduces decision latency from microseconds (typical of nftables) to nanoseconds. This ensures that legitimate queries are processed with minimal jitter, even when the server is under heavy load.

Optimization Proposed

Comparison: Before vs. After Scenarios

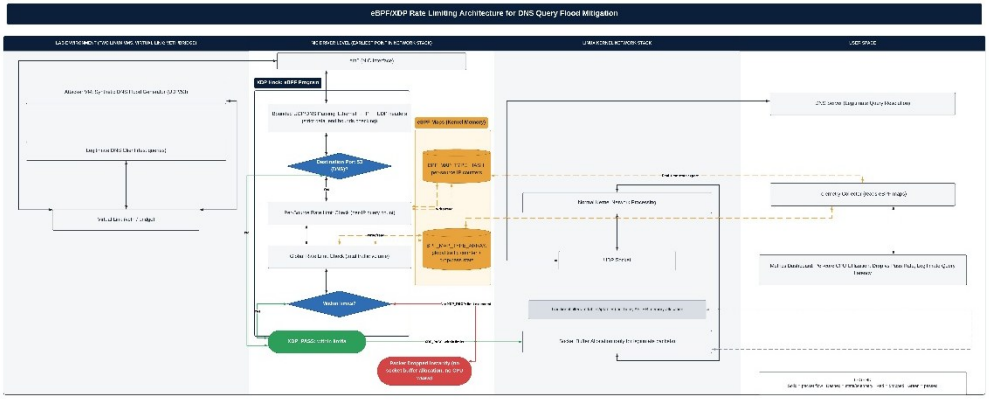
Feature	Before (No Filter / nftables)	After (eBPF/XDP Solution)	Technical Reason for Improvement
Processing Stage	After Kernel Memory Allocation (`sk_buff`)	**Before** Kernel Memory Allocation (NIC Driver)	XDP runs in the driver interrupt context, bypassing the network stack.
CPU Overhead	**High:** Wastes cycles allocating memory for drops	**Low:** Drops packets instantly; zero memory allocation	Eliminates `kmalloc` and softirq overhead for malicious traffic.
Latency	Microseconds (Kernel Stack Traversal)	**Nanoseconds** (Direct Driver Hook)	Direct hardware access vs. complex software stack traversal.
Attack Granularity	Coarse (Global rules only)	**Fine-Grained:** Per-IP tracking + Global cap	eBPF Maps allow stateful tracking of millions of unique IPs.

Optimization Proposed

Comparison: Before vs. After Scenarios

Feature	Before (No Filter / nftables)	After (eBPF/XDP Solution)	Technical Reason for Improvement
Legitimate Impact	High latency & drops during floods	**Minimal:** Legitimate traffic passes unaffected	Malicious packets are dropped before consuming CPU cycles needed for legit traffic.
Scalability	Limited by kernel `softirq` capacity	**Line-Rate:** Handles high-throughput efficiently	XDP is designed for line-rate processing (10Gbps+) with minimal CPU impact.
Safety/Stability	Risk of kernel panic from malformed packets	**High Stability:** Bounded parsing prevents OOB access	Strict bounds checking ensures the eBPF program never reads invalid memory.

Solution Architecture & Design



Data Flow Diagram

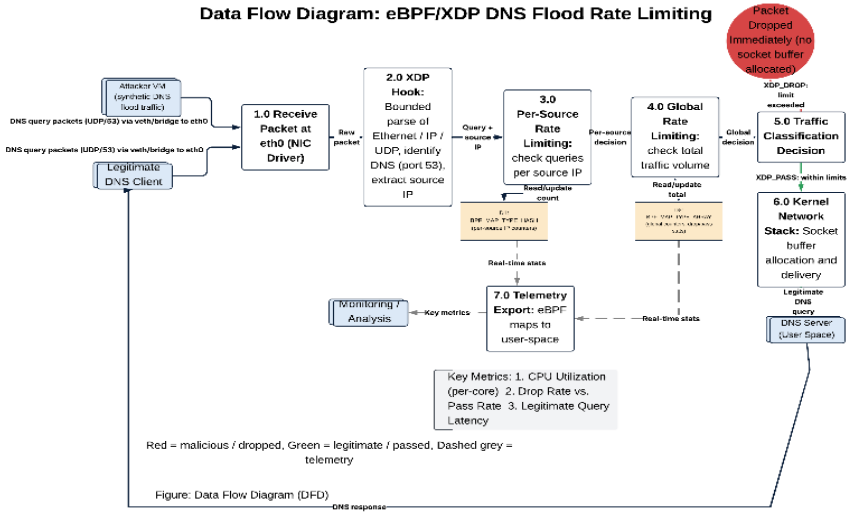


Figure: Data Flow Diagram (DFD)

Deployment Timeline

Methodology: Rapid Prototyping & Simulation Focus on architectural design, simulation-based validation, and comprehensive documentation.

Milestone 1: Design & Specification (Days 1–2)

- Define System Architecture & Data Flow Diagrams.
- Draft eBPF Pseudocode (Bounded Parsing, Hash/Array Maps).
- Deliverable: Technical Specification Document.

Milestone 2: Simulation Framework (Days 3–4)

- Set up Linux Host Pair (VMs) and Virtual Link.
- Develop Traffic Generation Harness (Python/Scapy).
- Configure Baselines (No-Filter & nftables).
- Deliverable: Functional simulation environment.

Milestone 3: Analysis & Visualization (Days 5–6)

- Run 3 Synthetic Load Scenarios against baselines.
- Collect CPU, Drop Rate, and Latency metrics.
- Project XDP performance gains using literature benchmarks.
- Deliverable: Performance Data & Graphs.

Deployment Timeline

Milestone 4: Reporting & Video (Days 7–8)

- Draft Technical Report sections.
- Record Demo Video (Architecture + Pseudocode + Results).
- **Deliverable:** Final PDF and Video File.

Milestone 5: Submission (Oct 14)

- Upload PDF/Video to GitHub.
- Transfer repository ownership to **`aiori-hackathon`**.
- **Deliverable:** Complete Submission Package

Bibliography I

1 [1] S. Kostopoulos, P. Tsakonas, and E. K. Markakis, "eBPF-based cybersecurity mechanisms: a systematic literature review," International Journal of Information Security, Springer, 2026. [Online]. Available: **https://doi.org/10.1007/s10207-026-01326-x** eBPF / XDP Foundations

2 [6] S. Gwak et al., "Container Instrumentation and Enforcement System for Runtime Security of Kubernetes Platform with eBPF," Intelligent Automation & Soft Computing, 2023. [Online]. Available: **https://doi.org/10.32604/iasc.2023.039565** DNS Security & RFCs

3 [7] RFC 8484: DNS Queries over HTTPS (DoH), IETF, 2018. [Online]. Available: **https://datatracker.ietf.org/doc/html/rfc8484**

4 [8] RFC 8482: DNS Transport over TLS - Implementation Requirements and Well-Known URI, IETF, 2018. [Online]. Available: **https://datatracker.ietf.org/doc/html/rfc8482**

Thank You for Your Attention!